

Module: Automated Vegetation Mapping using Google Earth Engine and Python

1. Introduction and Objectives

Satellite remote sensing plays a critical role in modern environmental monitoring, forestry management, and agricultural assessment. Traditionally, processing satellite imagery required downloading massive, multi-gigabyte data files and utilizing heavy desktop GIS software.

This workflow introduces an efficient, cloud-based alternative using **Google Earth Engine (GEE)** powered by Python in a **Google Colab** environment. By leveraging Google's cloud infrastructure, we can instantly query, filter, and analyze petabytes of geospatial data without taxing local computer hardware.

By completing this analysis manual, users will learn how to:

1. Establish a cloud-computing pipeline between Google Colab and the Google Earth Engine API.
2. Define a specific spatial **Area of Interest (AOI)** using geographic coordinates and distance buffers.
3. Query the **Copernicus Sentinel-2** satellite registry and apply cloud-cover and temporal filtering.
4. Calculate the **Normalized Difference Vegetation Index (NDVI)** to quantify canopy health.
5. Generate interactive geographic maps and extract localized summary statistics.

2. Methodology

Spatial Buffering

Rather than analyzing an entire country or arbitrary administrative boundary, this workflow isolates a localized study area. Starting from a single geographic point coordinate, the script constructs a **spatial buffer** a uniform circular zone extending radially outward by a specified distance (in this case, 25 kilometers). This acts as a targeted "cookie cutter" for all subsequent satellite data.

The Sentinel-2 Harmonized Dataset

This pipeline utilizes imagery from the European Space Agency's (ESA) **Sentinel-2** constellation. Specifically, it calls the **Surface Reflectance (SR) Harmonized** collection. Surface reflectance data has undergone rigorous atmospheric correction, removing distortions caused by gases, aerosols, and dust, ensuring that pixel values represent the true physical reflection of the Earth's surface.

Cloud Masking and Median Compositing

To deal with the persistent challenge of cloud cover in satellite imagery, the workflow employs a two-step reduction technique:

- **Quality Filtering:** Discards any individual satellite scene containing more than 20% cloud cover.
- **Temporal Median Compositing:** Stacked observations spanning an entire calendar year are evaluated pixel-by-pixel. The engine selects the statistical *median* value for each coordinate. This automatically eliminates transient anomalies like moving cloud remnants, terrain shadows, and sensor glitches, yielding a pristine, seamless composite image.

Normalized Difference Vegetation Index (NDVI)

The primary analytical output of this workflow is the calculation of **NDVI**, a standardized index utilized to estimate the abundance and vigor of green vegetation.

NDVI exploits the unique spectral signature of chlorophyll in living plants. Healthy vegetation strongly absorbs visible red light (for photosynthesis) and highly reflects near-infrared (NIR) light. The index mathematically normalizes this relationship:

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}}$$

The resulting values range strictly from **-1.0 to 1.0**:

- **Negative values to 0:** Indicate water, snow, ice, or cloud residue.
- **0 to 0.2:** Indicate bare soil, rock, or urban concrete.
- **0.2 to 0.5:** Indicate sparse vegetation, shrubs, or crops.
- **0.6 to 1.0:** Indicate dense, healthy, closed-canopy forests.

3. System Requirements & Environment Setup

To execute this script successfully, ensure you have the following prerequisites:

- A modern web browser with access to **Google Colab**.
- A registered **Google Earth Engine** developer account (free for research, education, and evaluation purposes).
- An active internet connection capable of communicating with GEE cloud servers.

Step 1. Library Installation (`!pip install...`)

- **What it does:** Tells Google Colab to run a command in the underlying Linux terminal rather than executing Python code.
- **earthengine-api:** This is the official Python client library used to communicate with Google's Earth Engine servers.
- **geemap:** A highly popular Python package designed specifically for interactive mapping with Google Earth Engine in Jupyter/Colab notebooks. It bridges the gap between GEE's data and interactive visualization tools.

Note: As the comment mentions, you only need to run this cell once per Colab session. If you restart your runtime, you will need to re-run it.

Step 2. Importing Libraries (`import...`)

- **ee:** Imports the Earth Engine ecosystem so you can call its data structures and algorithms (like `ee.Geometry`, `ee.ImageCollection`, etc.).
- **geemap:** Imports the mapping library so you can later create interactive maps, add layers, and visualize your spatial data.

Step 3. Authentication & Initialization (`ee.Authenticate()` & `ee.Initialize()`)

- **ee.Authenticate():** This triggers the authentication process. In Colab, it will usually prompt you with a link or a pop-up to log into your Google Account that has Earth Engine access granted. It generates an access token.
- **ee.Initialize():** This connects your current Python session to the Google Earth Engine servers using the authenticated token, allowing you to start requesting satellite imagery and spatial operations.

Tip: In modern Earth Engine environments, you often only need to run `ee.Initialize()` if you have already authenticated in a past session, but keeping both ensures no connection errors occur.

Step 4. Defining the Area of Interest (`tepi = ...`)

- **ee.Geometry.Point([35.42, 7.20]):** This creates a specific geographic point using coordinates. Earth Engine expects coordinates in [Longitude, Latitude] format.
 - *Context:* Coordinates [35.42, 7.20] point directly to **Tepi**, a town in southwestern Ethiopia.
- **.buffer(25000):** This creates a circular zone (a buffer zone) around that point. The units in Earth Engine buffers are **meters**.
 - $25,000 \text{ meters} = 25 \text{ kilometers}$.
- **tepi:** This variable now holds a circular polygon representing a 25 km radius around Tepi, which you can use to clip satellite images or filter datasets.

```

# Install required libraries (run once in Colab)
!pip install earthengine-api geemap

# Import libraries
import ee
import geemap

# Authenticate and initialize Earth Engine
ee.Authenticate()
ee.Initialize()

# -----
# 1. Define Tepi Area (25 km buffer)
# -----
tepi = ee.Geometry.Point([35.42, 7.20]).buffer(25000)

```

Step 5. Load Sentinel-2 Harmonized Dataset (`s2 = ...`)

This block queries Google's massive archive to pull a specific collection of satellite imagery and chains several filters together to narrow down the data:

- `ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")`: This calls the Sentinel-2 Surface Reflectance dataset. "Surface Reflectance" means the imagery has been corrected to remove atmospheric distortion (like haze or smoke), giving you the true color of the ground.
- `.filterBounds(tepi)`: This acts like a geographic cookie cutter. It tells Earth Engine to *only* look at satellite tiles that physically overlap with your 25 km Tepi buffer zone.
- `.filterDate('2023-01-01', '2023-12-31')`: This restricts the search to a one-year window (all of 2023).
- `.filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))`: This is crucial quality control. It discards any satellite images where cloud cover covers more than 20% of the tile. `lt` stands for "less than".
- `.select(['B4', 'B8'])`: To save memory and speed up processing, you are throwing away all the other color bands and only keeping two specific ones:
 - **B4**: The **Red** band.
 - **B8**: The **Near-Infrared (NIR)** band.

Step 6. Create Median Composite (`image = ...`)

Up until this line, `s2` is still a collection of multiple satellite images taken on different days throughout 2023. This line collapses that collection into a single, clean image.

- `.median()`: Earth Engine looks through the "stack" of images for the year 2023. For every single pixel location, it sorts all available cloud-free observations and picks the

median value. This is a standard remote sensing trick to automatically remove any leftover random cloud remnants, shadows, or sensor glitches.

- `.clip(tepi)`: The original satellite tiles are huge squares. This neatly trims the final median image down to the exact 25 km circular boundary around Tepi you defined earlier.

Step 7. Calculate NDVI (`ndvi = ...`)

This line calculates the **Normalized Difference Vegetation Index (NDVI)**, which is used to measure the health and density of green vegetation.

- `.normalizedDifference(['B8', 'B4'])`: This automatically mathematically computes the standard NDVI formula:

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}}$$

```
# 2. Load Sentinel-2 Harmonized Dataset
# -----
s2 = (ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")
      .filterBounds(tepi)
      .filterDate('2023-01-01', '2023-12-31')
      .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
      .select(['B4', 'B8'])) # Only required bands

# -----
# 3. Create Median Composite
# -----
image = s2.median().clip(tepi)

# -----
# 4. Calculate NDVI
# -----
ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')
```

Step 7. Initializing and Positioning the Map

- **Map = geemap.Map()**: This creates an empty, interactive map object powered by `geemap`. It behaves much like a Google Maps interface where you can zoom, pan, and toggle layers.
- **Map.centerObject(tepi, 10)**: This tells the map exactly where to focus when it loads.
 - It centers the view directly over your `tepi` geometry (the 25 km buffer area).
 - The number `10` represents the **zoom level**. Zoom levels in GIS usually range from 0 (the entire world) to 20+ (street level). Level 10 is a great choice here—it is zoomed in enough to see regional vegetation patterns but wide enough to fit your entire 25 km circle on screen.

Step 8. Setting up the Styling Rules (`ndvi_vis = ...`)

Raw NDVI data is just a grid of numbers between -1 and 1 . Without visualization parameters, it would just look like a flat gray image. This dictionary sets the rules for how those numbers are converted into colors:

- **'min': -1 and 'max': 1**: This defines the value range for stretching the colors. Anything at -1 or below gets the first color; anything at 1 or above gets the last color.
- **'palette': ['blue', 'white', 'green']**: This creates a continuous color gradient based on your range:
 - **Values near -1 (Blue)**: Typically represents water bodies or heavy cloud residue.
 - **Values near 0 (White)**: Typically represents bare soil, urban structures, or dead vegetation.
 - **Values near 1 (Green)**: Represents dense, healthy, living green vegetation (like the forests around Tepi).

Step 9. Adding the Data Layers (`Map.addLayer(...)`)

This overlays your data onto the map baseline. The function follows a specific structure:

```
Map.addLayer(data, styling_rules, layer_name).
```

- **Map.addLayer(ndvi, ndvi_vis, 'NDVI 2023 - Tepi')**: Adds your processed NDVI imagery using the blue-white-green color rules. In the map's interactive layer manager, this specific layer will be named `'NDVI 2023 - Tepi'`.
- **Map.addLayer(tepi, {'color': 'red'}, 'Tepi Area')**: This draws the actual 25 km buffer circle boundary on top of everything as a **red outline**. This makes it easy to see exactly where your study area begins and ends.

Step 10. Displaying the Output (`Map`)

- **Map**: Leaving the variable name by itself at the very end of the cell is the Python notebook way of saying *"render this object on screen right now"*. When you run this cell,

an interactive map widget will pop up right below it showing your red boundary circle filled with a color-coded map of Tepi's 2023 vegetation!

```
# 5. Display Map in Colab
# -----
Map = geemap.Map()

Map.centerObject(tepi, 10)

ndvi_vis = {
    'min': -1,
    'max': 1,
    'palette': ['blue', 'white', 'green']
}

Map.addLayer(ndvi, ndvi_vis, 'NDVI 2023 - Tepi')
Map.addLayer(tepi, {'color': 'red'}, 'Tepi Area')

Map
```

Step 11. Zonal Statistics (`ndvi.reduceRegion(...)`)

The `.reduceRegion()` function is Earth Engine terminology for performing **zonal statistics**. It takes an image layer containing thousands of pixels and compresses (reduces) them into a single summary number based on specific parameters:

- **reducer=ee.Reducer.mean()**: This tells the system *how* to aggregate the data. Instead of finding the maximum value, minimum value, or total sum, it calculates the mathematical average (mean) of all the pixel values.
- **geometry=tepi**: This sets the spatial boundary. It tells Earth Engine to grab only the pixels that fall inside your 25 km circle around Tepi, ignoring everything outside it.
- **scale=10**: This specifies the spatial resolution in **meters**. Since Sentinel-2 imagery has a base resolution of 10 meters per pixel for its Red and NIR bands, setting `scale=10` ensures that Earth Engine analyzes the data at its native, highest-quality resolution.
- **maxPixels=1e13**: This is a safety override parameter. Earth Engine has built-in guardrails to stop operations if a calculation involves too many pixels (to protect server resources). `1e13` (1×10^{13} , or 10 trillion pixels) raises this ceiling so that your calculation won't hit an accidental "too many pixels" error.

Step 12. Printing the Output (`print(..., mean_ndvi.getInfo())`)

- **mean_ndvi**: Up until this point, the calculations are still sitting lazily on Google's cloud servers as an abstract object.
- **.getInfo()**: This is the key that unlocks the data. It forces Earth Engine to actually execute the calculation on the server, transfer the resulting number over the internet, and bring it directly into your local Google Colab environment.

- `print(...)`: This displays the text string along with the final output. The console will print out a Python dictionary containing the average NDVI value for that entire area during 2023 (e.g., `{ 'NDVI': 0.65 }`).

```
mean_ndvi = ndvi.reduceRegion(
    reducer=ee.Reducer.mean(),
    geometry=tepi,
    scale=10,
    maxPixels=1e13
)

print("Mean NDVI value for Tepi:", mean_ndvi.getInfo())
```

What the Result Means

Once you run this cell, you will get a single value back:

- **Above 0.5:** Indicates highly dense, healthy canopy cover or dense forests (highly likely for the lush regions around Tepi).
- **0.2 to 0.5:** Indicates grassland, shrubs, or crops.
- **Close to 0:** Indicates bare soil or urban build-up.

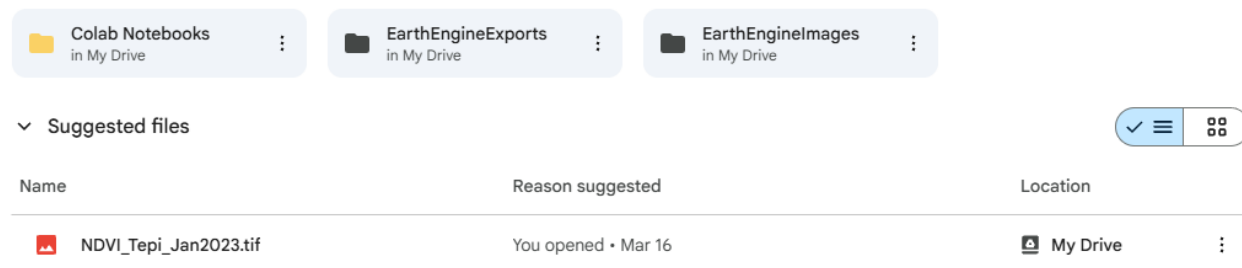


Figure 1: Successful export of the finalized NDVI raster dataset (`NDVI_Tepi_Jan2023.tif`) from Google Earth Engine saved directly into the user's Google Drive storage interface

Map Production via ArcGIS Desktop (ArcMap)

Step 1: Initialize the Project and Coordinate System

1. Launch **ArcMap** and select a new empty map template.
2. Open the **Catalog** window on the right side of the screen, right-click on *Folder Connections*, select *Connect to Folder*, and navigate to the directory where you downloaded your exported files from Google Drive.

3. Right-click on **Layers** in the *Table of Contents* (left pane) and choose **Properties**. Under the **Coordinate System** tab, verify or set the data frame coordinate system to **WGS 1984** to precisely match your Earth Engine environment data.

Step 2: Import Spatial Datasets

1. Drag and drop your exported `NDVI` GeoTIFF raster from the Catalog window directly into the map canvas. Click *yes* if prompted to build pyramids (this optimizes rendering performance).
2. If you exported the vector boundary geometry, drag and drop the `tepi` shapefile/feature class into the map layout as well.
3. Arrange your layers in the Table of Contents so that the vector boundary layer sits directly above the raster NDVI image layer.

Step 3: Configure Raster Symbolization and Colors

1. Right-click your NDVI raster layer in the Table of Contents and select **Properties**, then navigate to the **Symbology** tab.
2. Change the show method from *Unique Values* to **Stretched**.
3. Click on the **Color Ramp** dropdown menu. To match your interactive script visual output, choose or create a continuous tri-color ramp that transitions sequentially from **red** (minimum values) to **yellow** (neutral/mid values) to **Green** (maximum vegetation values).
4. Check the box for **Edit High/Low Values** or manually adjust the Min/Max stretch properties to **-1** and **1** respectively to match standard empirical biophysical classification scales.

Step 4: Switch to Layout View & Format Page Setup

1. Look at the bottom left toolbar of the map display window and toggle the view mode from *Data View* to **Layout View**.
2. Go to the top main menu file controls and navigate to **File > Page and Print Setup**.
3. Choose your desired output paper dimensions (e.g., Letter or A4) and set the structural orientation to **Landscape or portrait** to best match the geometry of a circular buffer grid layout. Click *OK*.
4. Use the **Select Elements** arrow tool to stretch the data frame layout box until it fits cleanly within the margins of your paper boundary grid lines.

Step 5: Insert Standard Map Elements

A scientifically sound map layout requires explicit reference elements. Use the primary **Insert** menu at the top of the window to embed the following components:

- **Title:** Insert a concise title block positioned prominently at the top (e.g., "*Spatial Distribution of Vegetation Health (NDVI) in Tepi, Ethiopia (2023)*").

- **Legend:** Insert a legend element. Follow the wizard steps to remove unwanted system layer names, ensuring it clearly shows the Red perimeter line as the "*Tepi Study Area*" and the color bar scaled from -1 to 1 labeled as "*NDVI Value*".
- **North Arrow:** Select an elegant, simple North Arrow graphic from the selector palette and place it neatly in an open corner space.
- **Scale Bar:** Insert a metric scale bar. Right-click the inserted element, choose *Properties*, and confirm that the *Scale Units* are explicitly configured to **Kilometers** or **Meters**.

Step 7: Export the Final Cartographic Asset

1. Verify that all components are aligned, legible, and uncrowded within the margins.
2. Navigate to **File > Export Map...** from the main controls menu.
3. In the export window, select your destination folder and change the *Save as type* format option to **PDF** or **TIFF** for optimal publishing quality.
4. Set the resolution value field to a minimum of **300 DPI** (Dots Per Inch) to guarantee that pixel details and text vectors remain perfectly sharp when printed or embedded into reports. Click **Save**.

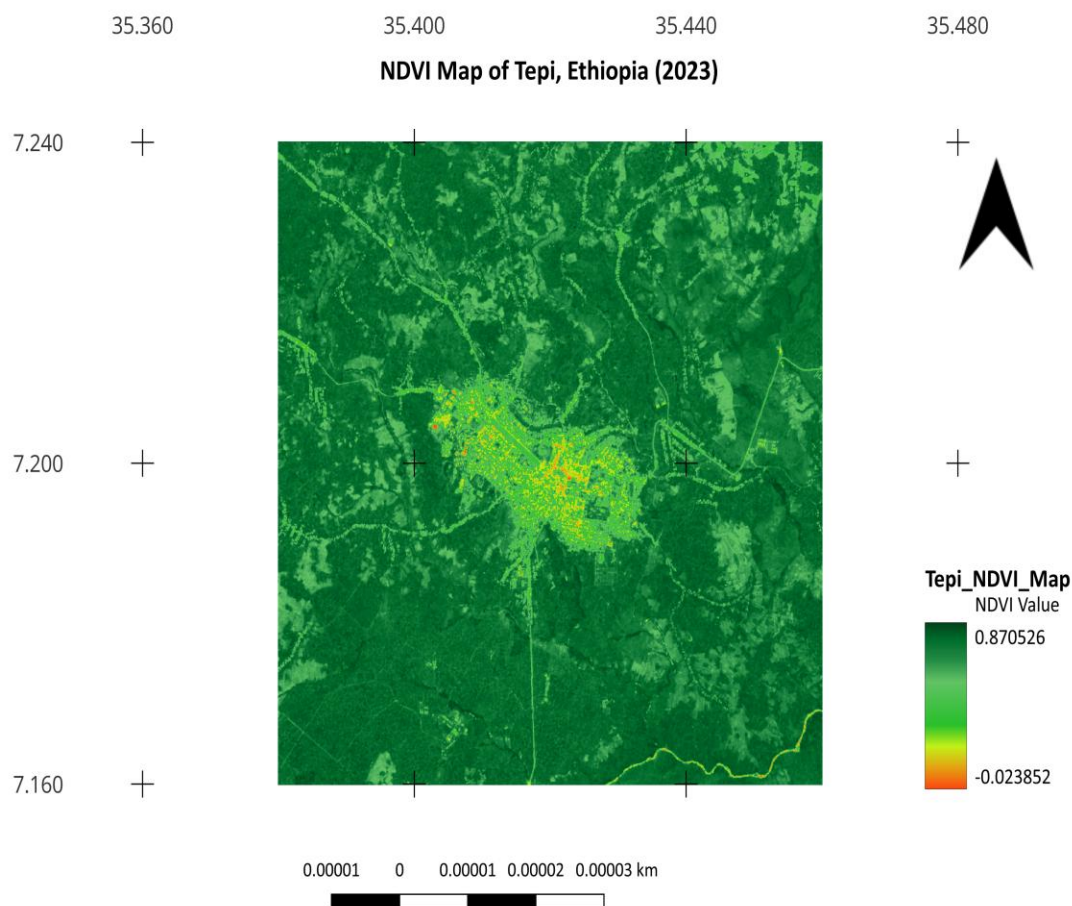


Figure 2. NDVI map of the Tepi study area derived from January 2023 satellite imagery, showing the spatial distribution of vegetation cover and health.

GIS meaning

1. Authenticate and Initialize

This step establishes a live connection between your local environment and Google's Cloud-GIS architecture where multi-petabyte satellite catalogs are hosted. Instead of traditionally downloading large geospatial files to a local drive, this process initializes a cloud-computing pipeline to process data remotely on Google's servers.

4. Define Tepi Area (Area of Interest)

This step defines the **Area of Interest (AOI)** or geographic study boundary for your environmental analysis. By assigning geographic coordinates to a specific location and applying a metric distance buffer, a 25km circular vector polygon is generated to restrict data computations strictly to the Tepi region rather than running global calculations.

5. Load Sentinel-2 Image Collection

This step queries a **Multi-Spectral Raster Catalog** to isolate specific geospatial imagery while discarding unusable data. The query filters the collection to match your exact spatial footprint, isolates the temporal window of interest, filters out heavy cloud cover, and retains only the **Red (B4)** and **Near-Infrared (B8)** spectral bands essential for monitoring plant biology.

Band	Meaning	Use
B4	Red band	vegetation absorption
B8	Near Infrared	vegetation reflection

These two bands are required to calculate **NDVI**.

6. Create Median Composite

This operation utilizes temporal data reduction to synthesize multiple scenes into a single, optimized environmental layer. By calculating the statistical median value for every individual pixel over an entire year, transient atmospheric interference—such as cloud fragments, terrain shadows, and sensor artifacts—is discarded, resulting in a pristine, cloud-free baseline map representing true surface conditions.

7. NDVI Calculation

This calculation executes a **Spectral Index Transformation** that converts raw reflection numbers into a single-band thematic raster showing vegetation health. By normalizing the difference between the Near-Infrared and Red bands, it quantifies canopy structural density, where negative values indicate water or clouds, values near zero indicate bare soil, and high decimal values denote healthy, dense chlorophyll activity.

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

NDVI value	Meaning
<0	water or clouds
0–0.2	bare soil
0.2–0.5	sparse vegetation
0.5–0.8	dense vegetation

8. Create Map

This step initializes an empty **GIS Map Canvas layout** within your web interface, mimicking the initialization of a fresh project viewport in desktop GIS software. It establishes the baseline platform required to visualize, interact with, and stack geographic layers.

9. Center Map

This function manages the **Spatial Viewport Extent** by instructing the map display to dynamically pan and zoom to a targeted region. Setting the map center directly over the study area at zoom level 10 optimizes the spatial frame to encompass the entire 25 km buffer area on screen.

10. Define Visualization

This step configures the **Thematic Cartography Styling Rules** for your continuous raster dataset. Because raw calculated index values are abstract decimals, this styling dictionary maps specific numeric ranges to a pseudo-color gradient, ensuring that very low vegetation appears blue, moderate areas appear white, and dense forest systems stand out in vibrant green.

11. Add NDVI Layer

This step renders the customized **Thematic Vegetation Layer** onto the map canvas layout, overlaying the processed index data directly onto the geographic coordinate grid. This provides immediate visual insights into the spatial distribution and variation of plant canopy vigor across the landscape.

12. Add Boundary

This step overlays a **Vector Geometry Feature** on top of your raster imagery to provide structural context to the layout. Displaying the study area's outer edge as a bright red outline allows analysts to clearly identify the geometric limits of the zone being observed.

13. Display Map

This step activates the interactive user interface, rendering the combined vector and raster map stack directly within your notebook. It turns abstract backend server calculations into a fully functional, navigable GIS visualization tool.

14. Export NDVI

This process handles **Inter-Software Data Portability** by preparing the finalized cloud data for external workflows. It packages the continuous index raster into a standard **GeoTIFF** file format—preserving its 10-meter spatial resolution and geographic coordinate system metadata—so that it can be downloaded and used directly in desktop applications like QGIS.

15. Mean NDVI Calculation

This final operation performs **Zonal Statistics** to extract quantitative, regional environmental metrics. By compiling every 10-meter cell value bounded within your vector study perimeter and computing their mathematical average, it produces an empirical baseline number that represents the total, objective vegetation density of the ecosystem.